DTypes.c

Line-by-Line Explanation for Beginners

How C converts between numbers and raw bytes

Overview — What DTypes.c Does

DTypes.c is a translation layer between two representations of the same data:

Numbers (float, int32_t)	⟷	Raw bytes (uint8_t array)
Human-readable values	⟷	What travels over serial / gets stored in flash
`3.14f`	⟷	`0x40 0x48 0xF5 0xC3`
`1000 (integer)`	⟷	`0xE8 0x03 0x00 0x00`

Key concept: Every float or int32_t is stored in memory as 4 bytes. This file provides functions to pack those bytes into a uint8_t array (for sending/storing) and to unpack them back into numbers (after receiving/loading).

The Complete File at a Glance

```
#define SOURCE_FILE "DTYPES"


#include <string.h>


#include "DTypes.h"

#include "Tensor.h"


/* ■■ 8 functions below ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

int32_t readBytesAsInt32(uint8_t *bytes) { ... }

int32_t readNumberOfBytesAsInt32(...) { ... }

void readBytesAsInt32Array(...) { ... }

float readBytesAsFloat(uint8_t *bytes) { ... }

void readBytesAsFloatArray(...) { ... }

void writeInt32ToByteArray(...) { ... }

void writeInt32ArrayToByteArray(...) { ... }

void writeFloatToByteArray(...) { ... }

void writeFloatArrayToByteArray(...) { ... }
```

Line 1 — #define SOURCE_FILE

`#define SOURCE_FILE "DTYPES"`	Gives this file an identity label

Every `.c` file in this project defines `SOURCE_FILE` as its own name. This macro is used by the debug-print macros in Common.h to show which file a message came from.

```
/* When PRINT_DEBUG runs inside this file, output looks like: */

[DTYPES: readBytesAsFloat] reading 4 bytes...

^^^^^

comes from SOURCE_FILE
```

Lines 3–5 — #include Statements

`#include <string.h>`	Provides memcpy() — the core tool used everywhere
`#include "DTypes.h"`	Our own header — function declarations
`#include "Tensor.h"`	Framework types used indirectly

Why string.h?

`string.h` is a standard C library header. It provides `memcpy()` — the only function DTypes.c actually calls. Every single function in this file relies on it. Without this include, the compiler would not know what memcpy is.

What is memcpy?

```
memcpy(destination, source, numberOfBytes)

/* copies numberOfBytes raw bytes from source to destination */

/* it does NOT care about types — just copies the bits as-is */
```

Key concept: memcpy is the correct and safe way to reinterpret bytes as a different type in C. Casting a pointer directly (e.g. *(int32_t*)bytes) is undefined behaviour — the compiler is allowed to produce wrong results. memcpy is always safe.

Deep Dive — How memcpy Converts Types

Before explaining each function, you must understand what memcpy actually does to memory. Here is a concrete example:

Scenario: converting float 3.14 to bytes

```
float x = 3.14f;


/* In RAM, x occupies 4 consecutive bytes: */

address of x: [0xC3] [0xF5] [0x48] [0x40]

[0] [1] [2] [3]


/* These bytes ARE 3.14 in IEEE 754 binary format */

/* They look like random hex but the CPU reads them as 3.14 */
```

memcpy copies those bytes without interpreting them:

```
uint8_t buf[4];

memcpy(buf, &x;, 4); /* copy 4 bytes FROM x INTO buf */


/* Now buf contains: */

buf[0] = 0xC3

buf[1] = 0xF5

buf[2] = 0x48

buf[3] = 0x40


/* And to convert back: */
```

```
float y;

memcpy(&y;, buf, 4); /* copy 4 bytes FROM buf INTO y */

/* y is now 3.14 again — exact same bits */
```

Tip: sizeof(float) == sizeof(int32_t) == 4 bytes on all platforms this project targets. sizeof() is always used instead of the literal 4 to make the code self-documenting and portable.

Function 1 — readBytesAsInt32

```
int32_t readBytesAsInt32(uint8_t *bytes)
```

Read 4 consecutive bytes from memory and return them as a signed 32-bit integer.

Parameters and return value

Parameter	Type	Direction	Meaning
`bytes`	`uint8_t *`	IN	Pointer to the start of at least 4 bytes to read
`return value`	`int32_t`	OUT	The 4 bytes reinterpreted as a signed 32-bit integer

The code — line by line

`int32_t readBytesAsInt32(uint8_t *bytes)`	Function header — name, parameter, return type
`{`	
`int32_t x;`	Reserve 4 bytes on the stack (uninitialised)
`memcpy(&x;, bytes, sizeof(int32_t));`	Copy 4 bytes from bytes into x
`return x;`	Return x as an int32_t number
`}`	

Syntax breakdown

```
memcpy( &x;, bytes, sizeof(int32_t) );

^ ^ ^

destination source how many bytes

address of x input pointer 4 (always)
```

`&x;` means "the address of x" — we pass memcpy a pointer to where x lives in memory, so it can write into x directly. Without the `&`, we would pass the *value* of x (which is uninitialised garbage) instead of its address.

Worked example

```
uint8_t data[4] = {0x01, 0x00, 0x00, 0x00};
```

```
int32_t result = readBytesAsInt32(data);

/* result = 1 (little-endian: lowest byte first) */


uint8_t data2[4] = {0xE8, 0x03, 0x00, 0x00};

int32_t result2 = readBytesAsInt32(data2);

/* result2 = 1000 (0x000003E8 = 1000) */
```

Function 2 — readNumberOfBytesAsInt32

`int32_t readNumberOfBytesAsInt32(uint8_t *data, size_t numberOfBytes)`

Read fewer than 4 bytes and zero-extend them into a full int32_t.

Parameters and return value

Parameter	Type	Direction	Meaning
`data`	`uint8_t *`	IN	Pointer to the bytes to read
`numberOfBytes`	`size_t`	IN	How many bytes to read (1, 2, or 3 — less than 4)
`return value`	`int32_t`	OUT	The bytes placed in the low end of a 32-bit integer, rest is zero

The code — line by line

`int32_t readNumberOfBytesAsInt32(uint8_t *data, size_t numberOfBytes) {`	Function header with 2 parameters
`int32_t output = 0;`	ALL 4 bytes start as zero: 0x00 0x00 0x00 0x00
`memcpy(&output;, data, numberOfBytes);`	Copy only N bytes — rest stays zero
`return output;`	Return the zero-extended result
`}`	

Why initialise output to 0?

Because we copy fewer than 4 bytes. If we only copy 2 bytes, the upper 2 bytes of `output` must be zero — not garbage. Initialising to 0 ensures the unwritten bytes are clean.

```
/* numberOfBytes = 2, data = {0x05, 0x01} */


output starts as: [0x00] [0x00] [0x00] [0x00]

after memcpy: [0x05] [0x01] [0x00] [0x00]

^^^^ ^^^^ ^^^^ ^^^^

copied copied zero zero


integer value: 0x00000105 = 261
```

Function 3 — readBytesAsInt32Array

```
void readBytesAsInt32Array(size_t numberOfValues, uint8_t *bytes, int32_t *outputArray)
```

Convert a flat byte stream into an array of int32_t values.

Parameters

Parameter	Type	Direction	Meaning
`numberOfValues`	`size_t`	IN	How many integers to read
`bytes`	`uint8_t *`	IN	Source: flat byte array — must be numberOfValues * 4 bytes long
`outputArray`	`int32_t *`	OUT	Destination: caller provides this array; function fills it
`return value`	`void`	—	Nothing — results written directly into outputArray

The code — line by line

`void readBytesAsInt32Array(`	void = no return value
`size_t numberOfValues,`	how many integers to read
`uint8_t *bytes,`	source byte stream
`int32_t *outputArray) {`	destination array
`for (size_t i = 0;`	i counts integers (0, 1, 2 ...)
`i < numberOfValues;`	stop after all integers done
`i++) {`	move to next integer
`size_t byteIndex =`	calculate where in bytes[] this integer starts
`i * sizeof(int32_t);`	integer i starts at byte i*4
`int32_t value =`	read 4 bytes starting at byteIndex
`readBytesAsInt32(`	calls Function 1
`&bytes;[byteIndex]);`	pass pointer to byte i*4
`outputArray[i] = value;`	store result in output slot i
`}`	

```
}
```

The key formula — byteIndex = i * sizeof(int32_t)

Each integer takes exactly 4 bytes. So integer 0 starts at byte 0, integer 1 starts at byte 4, integer 2 starts at byte 8, and so on.

```
bytes: [01 00 00 00] [02 00 00 00] [03 00 00 00]

^^^^^^^^^^^ ^^^^^^^^^^^ ^^^^^^^^^^^

integer 0 integer 1 integer 2

byteIndex=0 byteIndex=4 byteIndex=8


outputArray: [1] [2] [3]
```

Function 4 — readBytesAsFloat

```
float readBytesAsFloat(uint8_t *bytes)
```

Read 4 consecutive bytes and return them as a float — identical pattern to Function 1.

Parameters and return value

Parameter	Type	Direction	Meaning
`bytes`	`uint8_t *`	IN	Pointer to the start of exactly 4 bytes
`return value`	`float`	OUT	The 4 bytes reinterpreted as an IEEE 754 float

The code — line by line

```
float readBytesAsFloat(uint8_t *bytes) {

float x;

memcpy(&x;, bytes, sizeof(float));

return x;

}
```

Returns float, not int32_t — only difference

Reserve 4 bytes for a float

Copy 4 bytes into x — same as Function 1

Return as float

Same bytes — completely different meaning

```
uint8_t buf[4] = {0x00, 0x00, 0x80, 0x3F};


int32_t as_int = readBytesAsInt32(buf); /* = 1,065,353,216 */

float as_float = readBytesAsFloat(buf); /* = 1.0f */


/* SAME 4 bytes, completely different values depending on type */
```

Note: This is why the type system matters. The bytes have no inherent meaning — the meaning comes from how you interpret them. DTypes gives you explicit control over which interpretation to use.

Function 5 — readBytesAsFloatArray

```
void readBytesAsFloatArray(size_t numberOfValues, uint8_t *bytes, float *outputArray)
```

Convert a flat byte stream into an array of floats — mirrors Function 3 exactly.

The code — line by line

`void readBytesAsFloatArray(`	
`size_t numberOfValues,`	how many floats to read
`uint8_t *bytes,`	source: flat byte stream
`float *outputArray) {`	destination: caller's float array
`for (size_t i = 0; i < numberOfValues; i++) {`	loop over every float
`size_t byteIndex = i * sizeof(float);`	float i starts at byte i*4
`float value = readBytesAsFloat(`	call Function 4
`&bytes;[byteIndex]);`	pass pointer to the right 4 bytes
`outputArray[i] = value;`	store result
`}`	
`}`	

Real-world use in this project:

```
/* After receiving a serialized weight tensor over UART: */

uint8_t received[4000]; /* 1000 floats * 4 bytes each */

float weights[1000];

readBytesAsFloatArray(1000, received, weights);

/* weights[] now contains all 1000 float values, ready to use */
```

Function 6 — writeInt32ToByteArray

```
void writeInt32ToByteArray(int32_t value, uint8_t *bytes)
```

Exact reverse of Function 1 — takes one int32_t and writes its 4 bytes into an array.

Parameters

Parameter	Type	Direction	Meaning
`value`	`int32_t`	IN	The integer you want to convert to bytes
`bytes`	`uint8_t *`	OUT	Destination: caller provides this, must have room for 4 bytes
`return value`	`void`	—	Nothing — bytes written directly into the bytes[] array

The code — line by line

`void writeInt32ToByteArray(int32_t value, uint8_t *bytes) {`	void — no return value
`memcpy(bytes, &value;, sizeof(int32_t));`	copy 4 bytes FROM value INTO bytes[]
`}`	

Notice the argument order flip vs reading:

```
/* READING — bytes is SOURCE, &x; is DESTINATION */

memcpy(&x;, bytes, sizeof(int32_t));

^^^ ^^^^^

dest source


/* WRITING — bytes is DESTINATION, &value; is SOURCE */

memcpy(bytes, &value;, sizeof(int32_t));

^^^^^ ^^^^^^

dest source
```

Warning: memcpy always goes from right to left: memcpy(destination, source, size). Getting the order wrong silently corrupts memory — no compiler warning.

Worked example

```
uint8_t buf[4];

writeInt32ToByteArray(1000, buf);


/* buf now contains: */

buf[0] = 0xE8 /* least significant byte */

buf[1] = 0x03

buf[2] = 0x00

buf[3] = 0x00 /* most significant byte */


/* 0x000003E8 = 1000 in little-endian */
```

Function 7 — writeInt32ArrayToByteArray

```
void writeInt32ArrayToByteArray(size_t numberOfValues, int32_t *valueArray, uint8_t *bytes)
```

Serialize an entire array of int32_t values into a flat byte stream.

Parameters

Parameter	Type	Direction	Meaning
`numberOfValues`	`size_t`	IN	Number of integers to write
`valueArray`	`int32_t *`	IN	Source: the integer array to serialize
`bytes`	`uint8_t *`	OUT	Destination: must be numberOfValues * 4 bytes large
`return value`	`void`	—	Nothing — fills bytes[] directly

The code — line by line

`void writeInt32ArrayToByteArray(`	
`size_t numberOfValues,`	loop count
`int32_t *valueArray,`	source integers
`uint8_t *bytes) {`	destination bytes
`for (size_t i = 0; i < numberOfValues; i++) {`	loop over each integer
`size_t byteIndex = i * sizeof(int32_t);`	slot i starts at byte i*4
`memcpy(`	copy one integer to its slot
`&bytes;[byteIndex],`	destination: address of slot i in byte array
`&valueArray;[i],`	source: address of integer i
`sizeof(int32_t));`	4 bytes
`}`	
`}`	

What &bytes;[byteIndex] means:

```
bytes[byteIndex] /* the VALUE at position byteIndex */

&bytes;[byteIndex] /* the ADDRESS of position byteIndex */

/* memcpy needs an address, not a value */


/* Equivalently written as: */

bytes + byteIndex /* pointer arithmetic — same result */
```

Functions 8 & 9 — writeFloatToByteArray / writeFloatArrayToByteArray

These are the float counterparts of Functions 6 and 7. The code is structurally identical — only the type changes from `int32_t` to `float`.

`void writeFloatToByteArray(float value, uint8_t *bytes) {`	float instead of int32_t
`memcpy(bytes, &value;, sizeof(float));`	identical to Function 6
`}`	
`void writeFloatArrayToByteArray(`	
`size_t numberOfValues,`	
`float *valueArray,`	float* instead of int32_t*
`uint8_t *bytes) {`	
`for (size_t i = 0; i < numberOfValues; i++) {`	
`size_t byteIndex = i * sizeof(float);`	sizeof(float) = 4, same as int32_t
`memcpy(&bytes;[byteIndex],`	
`&valueArray;[i],`	
`sizeof(float));`	
`}`	
`}`	

Real-world use — serializing trained weights:

```
float trained_weights[384]; /* learned by training */

uint8_t byte_stream[384 * 4]; /* 1536 bytes */


writeFloatArrayToByteArray(384, trained_weights, byte_stream);

/* byte_stream is now ready to send over UART to the STM32 */


/* On the STM32 side: */

float restored_weights[384];

readBytesAsFloatArray(384, byte_stream, restored_weights);

/* restored_weights is identical to trained_weights */
```

Summary — The 8 Functions as Matched Pairs

Read (bytes → number)	Write (number → bytes)	Type	Single/Array
readBytesAsInt32()	writeInt32ToByteArray()	int32_t	Single
readNumberOfBytesAsInt32()	— (no write counterpart)	int32_t	Partial
readBytesAsInt32Array()	writeInt32ArrayToByteArray()	int32_t	Array
readBytesAsFloat()	writeFloatToByteArray()	float	Single
readBytesAsFloatArray()	writeFloatArrayToByteArray()	float	Array

Key concept: The entire file is built on one tool: memcpy(). All 8 functions are thin wrappers that call memcpy with the right source, destination, and size — either once (single) or in a loop (array). Understanding memcpy means understanding this whole file.